

Networking, HTTP Internals & Serving Cheatsheet

Lessons: [63 — Networking Fundamentals](#) · [64 — HTTP & net/http Internals](#) · [65 — Docker Networking](#) · [66 — Every Way to Serve a Go App](#)

Examples: [63](#) · [64](#)

Covers: TCP/IP, the `net` package, the HTTP wire format, TLS, the server machine, container networking, listeners

Legend: [*] = API the lessons have not covered yet

THE FOUR LAYERS

link	Ethernet/wifi frames, MAC addresses
internet	IP packets, addresses, routing, TTL
transport	TCP (ordered, reliable, a stream) UDP (messages, best effort)
application	HTTP, gRPC, DNS, TLS-wrapped anything

an address is not a connection a connection is a 4-TUPLE:
(src IP, src port, dst IP, dst port)
– which is how one server port serves 10,000 clients

TCP

3-way handshake	SYN -> SYN-ACK -> ACK (one round trip before any data)
byte stream, NO MESSAGE BOUNDARIES	the single most important fact. Two Writes can arrive as one Read, or as three.
framing is YOUR job	delimiter (<code>\n</code>) or length-prefix (4 bytes, then the body)
close	FIN -> ACK each way; half-open is legal (<code>CloseWrite</code>)
TIME_WAIT	2 x MSL on the side that closed FIRST; it's correct behaviour, not a leak
accept queue	the kernel completes handshakes into a backlog; your <code>Accept()</code> loop drains it. A slow loop = refused connections.
keepalive	detects a peer that vanished without a FIN
Nagle / delayed ACK	small writes get coalesced; <code>SetNoDelay(true)</code> for latency
MTU ~1500	bigger payloads fragment; PMTU discovery finds the limit
NAT	rewrites the tuple; that's why inbound needs a mapping

SOCKETS & THE GO RUNTIME

a socket is a file descriptor	<code>ulimit -n</code> is the real ceiling on connections
netpoller	the runtime multiplexes ALL sockets over <code>epoll/kqueue</code>
goroutine-per-connection	is cheap BECAUSE of that: a blocked goroutine costs ~4KB of stack, not an OS thread
so: write blocking code	and let the runtime make it async

THE net PACKAGE

```
net.Listen("tcp", ":8080")    -> (Listener, error)
ln.Accept()                  -> (Conn, error); the loop
ln.Addr()                    with ":0", the OS-assigned port – the test idiom
net.Dial("tcp", "host:port") -> (Conn, error)
net.DialTimeout(...)         [*] bound the connect
&net.Dialer{Timeout, KeepAlive}.DialContext(ctx, ...) the cancellable form
conn.Read(b) / conn.Write(b) Write can be PARTIAL – check n
conn.Close()
conn.SetDeadline / SetReadDeadline / SetWriteDeadline

                                THE ONLY cancellation a socket has. ctx does not
                                interrupt a blocked Read – a deadline does.

net.SplitHostPort / JoinHostPort never split on ":" yourself (IPv6)
net.ListenPacket("udp", ...) UDP: messages KEEP their boundaries, may be lost/reordered
net.Listen("unix", "/tmp/x.sock") Unix domain socket: no TCP stack, filesystem perms
net.Pipe()                    [*] an in-memory Conn – perfect for tests
net.LookupHost / Resolver     DNS
GODEBUG=netdns=go|cgo|2     [*] pick and debug the resolver
```

READING THE ERRORS

connection refused	nothing is listening on that port (fast, definitive)
i/o timeout	your deadline fired
connection reset by peer	the other side sent RST – it crashed or closed abruptly
EOF	a clean close by the peer, mid-read
no such host	DNS failure
address already in use	another process holds the port (or TIME_WAIT + no reuse)
broken pipe	you wrote to a connection the peer already closed
context deadline exceeded	your ctx, not the socket
toolbox	ss -tlnp / lsof -i / nc -zv / dig / tcpdump / curl -v

HTTP/1.1 ON THE WIRE

GET /path HTTP/1.1\r\n	the request line
Host: example.com\r\n	mandatory in 1.1
Header: value\r\n	
\r\n	a blank line ends the headers
[body]	
how the body's end is found	Content-Length: n – exactly n bytes Transfer-Encoding: chunked – size in hex, CRLF, data, ..., then a 0-length chunk neither, on a response: read until close
keep-alive	the default in 1.1; the connection is reused
head-of-line blocking	one slow response blocks the next on the SAME connection
pipelining	theoretically allowed, practically dead

HTTP/2 & HTTP/3

HTTP/2	binary FRAMES over one TCP connection
streams	many concurrent requests, multiplexed, interleaved
HPACK	header compression with a shared dynamic table
flow control	per stream and per connection
server push	deprecated in practice
still has TCP H0L blocking	one lost packet stalls every stream
HTTP/3 over QUIC	UDP-based; streams are independent, so no TCP H0L
	0-RTT resumption, connection migration across networks
h2c	[*] HTTP/2 without TLS – for internal hops only
Go's server does h2	automatically over TLS; the client too

TLS

handshake	ClientHello -> ServerHello + certificate -> key exchange (1-RTT in TLS 1.3, 2 in 1.2)
certificate chain	leaf -> intermediate -> root; SERVE THE INTERMEDIATES
SNI	the hostname in the ClientHello – how one IP serves many certificates
ALPN	negotiates the protocol IN the handshake: "h2" or "http/1.1" – this is where HTTP/2 comes from
mTLS	the client presents a certificate too; service identity
<code>tls.Config{MinVersion: tls.VersionTLS12}</code>	
<code>InsecureSkipVerify</code>	a deliberate MITM hole; never in production

THE net/http SERVER MACHINE

<code>ListenAndServe</code>	<code>net.Listen</code> , then <code>Serve(ln)</code>
<code>Serve(ln)</code>	<code>for { c, _ := ln.Accept(); go c.serve() }</code>
<code>go c.serve()</code>	ONE GOROUTINE PER CONNECTION – the whole design read request -> <code>ServeMux</code> -> your handler -> write -> loop for the next keep-alive request
<code>ServeMux</code>	pattern match -> <code>Handler</code>
<code>ResponseWriter</code>	buffered; the status is sent with the first flush
the five timeouts	
<code>ReadHeaderTimeout</code>	headers only – the Slowloris defence
<code>ReadTimeout</code>	headers + body
<code>WriteTimeout</code>	from the END of the request read to the end of the write – it BREAKS SSE/streaming; set it to 0 for those routes
<code>IdleTimeout</code>	keep-alive idle
Handler-level	<code>http.TimeoutHandler</code> , or a ctx deadline
<code>Flusher</code>	<code>w.(http.Flusher).Flush()</code> – push what's buffered
<code>Hijacker</code>	take over the raw connection (<code>WebSocket</code>)
<code>ResponseController</code>	[*] Go 1.20+: <code>SetWriteDeadline/Flush</code> per response – the modern replacement for the interface assertions
<code>CloseNotifier</code>	deprecated; use <code>r.Context()</code>

THE net/http CLIENT

http.Transport IS A CONNECTION POOL	one per client; reuse the client
MaxIdleConns / MaxIdleConnsPerHost	the default per-host is 2 – raise it for a service you call constantly
IdleConnTimeout / TLSHandshakeTimeout / ExpectContinueTimeout	
DisableKeepAlives	almost always wrong
THE DRAIN-AND-CLOSE RULE	<code>io.Copy(io.Discard, resp.Body); resp.Body.Close()</code> – without draining, the connection is NOT reused
one client, reused	a new <code>http.Client</code> per request = a new pool = socket exhaustion under load
retries	only for idempotent requests; the body must be replayable
X-Forwarded-For / -Proto	trust them ONLY from your own proxy, which must overwrite rather than append what the client sent
<code>httputil.ReverseProxy</code>	a production-grade proxy in a few lines; set <code>Director</code> or <code>Rewrite</code> , and handle <code>ErrorHandler</code>

CONTAINER NETWORKING

network namespace	each container has its own interfaces, routes, and ports
veth + bridge + NAT	the default bridge network's plumbing
-p 8080:8080	a DNAT rule on the host
EXPOSE	documentation. It does nothing.
KILLER #1	bind <code>0.0.0.0</code> , NOT <code>127.0.0.1</code> – localhost inside the container is unreachable from outside it
KILLER #2	"localhost" inside a container means THAT container
container DNS <code>127.0.0.11</code>	resolves service names on a user-defined network (the default bridge has NO DNS – use <code>compose networks</code>)
address a peer	<code>http://api:8080</code> – the SERVICE name and the CONTAINER port, never the published host port
<code>compose depends_on</code>	start order, NOT readiness – add a healthcheck + condition
<code>internal: true</code>	a network with no outbound route
scaling	multiple A records for one service name
host networking	no namespace, no NAT – Linux only
<code>host.docker.internal</code>	reach the host from a container (Docker Desktop)
scratch images	no CA certs (HTTPS fails), no <code>tzdata</code> , no <code>/etc/hosts</code> tools
PID 1	your process gets no default signal handlers – handle <code>SIGTERM</code> yourself; use <code>exec-form ENTRYPOINT</code>
debugging a toolless image	<code>docker run --network container:<id> nicolaka/netshoot</code>
Kubernetes mapping	pod = one netns shared by its containers; Service = a virtual IP; cluster DNS = <code>svc.namespace.svc.cluster.local</code>

EVERY WAY TO SERVE (the catalog)

the one primitive	<code>ln, _ := net.Listen(...); srv.Serve(ln)</code> – everything below is a different Listener
<code>:0</code>	an OS-assigned port; <code>ln.Addr()</code> tells you which
TLS	<code>srv.ListenAndServeTLS</code> / <code>tls.NewListener</code> / <code>autocert</code>
<code>netutil.LimitListener(ln, n)</code>	[*] cap concurrent connections
Unix socket	<code>net.Listen("unix", path)</code> – nginx in front, no TCP
systemd socket activation	the listener arrives as fd 3: <code>net.FileListener(os.NewFile(3, ""))</code>
<code>SO_REUSEPORT</code>	[*] several processes on one port; kernel load balancing
zero-downtime restart	(1) <code>SO_REUSEPORT</code> , (2) pass the fd to a child, (3) systemd socket activation, (4) a proxy in front
h2c	[*] HTTP/2 cleartext, for internal service hops
HTTP/3	[*] quic-go + an <code>Alt-Svc</code> header advertising it
raw TCP / UDP	your own protocol, straight on <code>net.Conn/PacketConn</code>
gRPC + REST on one port	switch on <code>Content-Type</code> + <code>ProtoMajor</code> , or use <code>cmux</code>
grpc bufconn	an in-memory gRPC listener for tests
behind a reverse proxy	nginx/Caddy/Traefik terminate TLS; trust <code>X-Forwarded-*</code>
<code>httputil.ReverseProxy</code>	be the proxy yourself
FastCGI / CGI	<code>net/http/fcgi</code> and <code>net/http/cgi</code> – legacy hosting
PaaS / Lambda	read <code>\$PORT</code> ; no background goroutines after the response
several servers, one process	<code>public :8080 + admin :9090</code> (pprof, metrics, health) – never expose pprof publicly
httptest / <code>net.Pipe</code>	no sockets at all, for tests

TRAPS & MEMORIZE

assuming <code>Read</code> returns a message	TCP has no boundaries; frame it yourself
ignoring the <code>n</code> from <code>Write</code>	partial writes are legal
no deadlines on a raw <code>Conn</code>	<code>ctx</code> will not save you; a <code>Read</code> blocks forever
one <code>Accept</code> loop doing work	<code>accept</code> , then <code>`go handle(conn)`</code> – always
binding <code>127.0.0.1</code> in Docker	the container is unreachable and nothing logs why
publishing <code>!= reachable</code>	<code>-p</code> maps the <code>HOST</code> ; containers talk on the container port
<code>depends_on</code> as readiness	the app starts before the database can accept
<code>WriteTimeout</code> on an SSE route	the server cuts the stream at the deadline
no <code>ReadHeaderTimeout</code>	Slowloris holds every connection open
a client per request	the pool is <code>per-Transport</code> ; you just disabled it
not draining <code>resp.Body</code>	the connection is closed instead of reused
trusting <code>X-Forwarded-For</code>	spoofable unless your proxy overwrites it
missing intermediates	works in your browser, fails in curl and in Go
exposing pprof publicly	heap dumps and goroutine stacks to anyone who asks